

Project Workflow

OptiCrop: Smart Agricultural Production Optimization Engine

This document describes the end-to-end workflow of the project, from problem definition to deployment, along with the tools used at each stage.

1. Define Problem and Understanding

Identify the agricultural problem — recommending the most suitable crop based on soil nutrients (N, P, K) and climate conditions (temperature, humidity, pH, rainfall). Gather business requirements and review existing crop-recommendation approaches.

Tools used: None (research & documentation stage).

2. Data Collection and Analysis

Collect the agricultural dataset from a reliable source and load it for exploration.

- Import libraries and read the dataset.
- Univariate analysis — distribution of individual features (histograms, boxplots).
- Bivariate analysis — relationship between each feature and crop suitability (e.g. rainfall by crop, pH by crop).
- Multivariate analysis — correlation heatmaps and pairplots to study interactions between multiple variables.

Tools used: Pandas (load/explore data), Matplotlib & Seaborn (visualizations).

3. Data Pre-processing

Prepare the dataset for modeling.

- Check for and handle missing values / duplicates.
- Detect and treat outliers.
- Engineer relevant features (e.g. seasonal indicators).
- Split the dataset into training and testing sets.

Tools used: Pandas, NumPy, Scikit-learn (train_test_split, scalers).

4. Model Building

Train and evaluate machine learning models.

- Apply K-Means Clustering to group similar agricultural/soil conditions.
- Train a Logistic Regression model to predict the most suitable crop from environmental parameters.
- Evaluate performance using accuracy, precision, recall, and confusion matrix.
- Save the best-performing model for deployment.

Tools used: Scikit-learn, NumPy, Pickle (model saving).

5. Application Building

Build a user-facing application around the trained model.

- Design an HTML front end where users input N, P, K, temperature, humidity, pH, and rainfall values.
- Build the Flask backend to load the saved model and return a crop recommendation.
- Run, test, and validate the app end-to-end for accurate predictions.

Tools used: Flask, HTML/CSS, Pickle (loading the saved model).

6. Conclusion

Summarize results, model accuracy, and how the application helps farmers/stakeholders choose the optimal crop for their land conditions.

Environment used throughout

Developed using Anaconda Navigator (environment/package management) and PyCharm (IDE) for writing, testing, and debugging all the above stages.